

```
@page { margin: 14mm 12mm; } body { font-family: "DejaVu Sans",
Arial, sans-serif; font-size: 10.5pt; line-height: 1.34; color: #1a2733;
max-width: 100%; margin: 0; } h1 { font-size: 19pt; color: #12324b;
border-bottom: 2px solid #12324b; padding-bottom: 3px; } h2 { font-
size: 14pt; color: #12324b; border-left: 4px solid #12324b; padding-
left: 7px; margin-top: 16px; } h3 { font-size: 12pt; color: #234;
margin-top: 12px; } table { border-collapse: collapse; width: 100%;
margin: 8px 0; font-size: 9pt; } th, td { border: 1px solid #b7c2cc;
padding: 3px 6px; text-align: left; vertical-align: top; } th {
background: #12324b; color: #fff; } tr:nth-child(even) td {
background: #f2f5f7; } code { background: #eef1f4; padding: 1px
3px; border-radius: 3px; font-size: 8.8pt; } pre { background:
#0f1b26; color: #dfe7ee; padding: 8px 10px; border-radius: 5px; font-
size: 8.4pt; overflow-x: auto; white-space: pre-wrap; } pre code {
background: transparent; color: inherit; }
```

README

O-RAN VIP Suite — README (run / regress / waves / coverage, all simulators)

Author: AVIK MAJUMDAR **Project:** AVIK VIP FACTORY — O-RAN VIP Suite **2026-08-09**

13-component O-RAN VIP: 4G LTE (CPRI-over-Ethernet) + 5G NR (eCPRI Split 7.2x). Includes a native-UVM **RAL (uvm_reg) register model** for O-RAN M-plane RU config. **Combined executed regression: 1523/1523 = 100% real pass** (1470 Lane-1 + 39 Lane-2A + 13 Lane-2B + 1 RAL).

Lanes and simulator support

Lane	What	Verilator 5.020	Verilator 5.050	VCS	Questa	Xcelium
Lane-1	plain-SV VIP (Z3 stimulus)	YES	YES	-	-	-
Lane-2A	UVM-subset (muvvm_pkg)	YES	YES	-	-	-
Lane-2B	native Accellera UVM	-	YES	YES	YES	YES
RAL	uvm_reg register model	-	YES	YES	YES	YES

Lane-1 / Lane-2A are Verilator-native (external Z3 stimulus + custom sim_main). Lane-2B is standard Accellera UVM and runs on Verilator 5.050 **and** any commercial sim. Everything is driven by make (see make help) or ./compile.sh <sim> <lane> [component].

0. Toolchain bring-up

```
make verilator050          # build Verilator 5.050 from source (git
v5.050, ~25 min)
make uvm-setup             # clone + patch Accellera uvm-core (DPI
```

```
shim)
export UVM_HOME=/tmp/uvmtest/uvm
```

Verilator 5.020 (apt) is sufficient for Lane-1 / Lane-2A.

1. STANDALONE run (single component / test)

Verilator (Lane-1)

```
make run LANE=1 COMPONENT=uplane TEST=smoke SEED=1
# or directly:
cd components/uplane && ./run.sh smoke 1 300
```

Verilator 5.050 (Lane-2B native UVM)

```
make run LANE=2b SIM=verilator5050 COMPONENT=uplane SEED=1
# or directly (after `make lane2b SIM=verilator5050` built the
binary):
cd lane2/lane2b_native/suite && ./obj_suite2/simv_uvm
+UVM_TESTNAME=uplane_test +ntb_random_seed=1
```

VCS (Lane-2B)

```
make run LANE=2b SIM=vcs COMPONENT=ecpri_transport SEED=1
# or directly:
cd lane2/lane2b_native/ecpri_transport
vcs -sverilog -full64 -ntb_opts uvm-1.2 -timescale=1ns/1ps -f
ecpri.f -o simv_vcs
./simv_vcs +UVM_TESTNAME=ecpri_test +ntb_random_seed=1
```

Questa (Lane-2B)

```
make run LANE=2b SIM=questa COMPONENT=ecpri_transport SEED=1
# or directly:
cd lane2/lane2b_native/ecpri_transport
qverilog -sv -mfcu -timescale 1ns/1ps +incdir+$UVM_HOME/src
$UVM_HOME/src/uvm_pkg.sv -f ecpri.f
vsim -c -do "run -all; quit" work.ecpri_tb_top
+UVM_TESTNAME=ecpri_test -sv_seed 1
```

Xcelium (Lane-2B)

```
make run LANE=2b SIM=xcelium COMPONENT=ecpri_transport SEED=1
# or directly:
cd lane2/lane2b_native/ecpri_transport
xrun -uvm -sv -timescale 1ns/1ps -f ecpri.f +UVM_TESTNAME=ecpri_test
-svseed 1
```

RAL — register model (uvm_reg, native UVM)

```
make ral SIM=verilator5050 # build + run oran_ral_test ->
PASS (163 ops, 0 err)
# or directly:
cd lane2/ral && UVM_HOME=/tmp/uvmtest/uvm bash build_ral.sh
./obj_ral/simv_ral +UVM_TESTNAME=oran_ral_test +ntb_random_seed=1
# VCS/Questa/Xcelium: compile oran_ral_pkg.sv + oran_ral_tb_top.sv
with UVM, same +UVM_TESTNAME
```

Register map: ru_ctrl@0x00 RW · ru_version@0x04 RO · ru_numant@0x08 RW
· ru_comp@0x0C RW · ru_bwp@0x10 RW. Checks: reset-value mirror, RO
version read-back, 20× RW randomize→update→read→mirror-
compare, RO write-protect. Front-door via uvm_reg_adapter
(reg2bus/bus2reg) + auto-predict.

2. REGRESSION

Verilator (Lane-1 + Lane-2A)

```
make lane1                # build + gates 2-8 + corner +
coverage + report
make lane2a                # UVM-subset, all 13 (seeds
1,2,3)
make regress-l1 COMPONENT=all # Lane-1 smoke (seeds 1,2,3)
make regress-l5 RAND=100     # Lane-1 random L3 + negative +
coverage
```

Verilator 5.050 (Lane-2B — all 13 native UVM)

```
make lane2b SIM=verilator5050 # build once + run 13 tests ->
13/13 PASS
```

VCS / Questa / Xcelium (Lane-2B + RAL)

```
make lane2b SIM=vcs        # (or questa | xcelium) ->
run_commercial.sh (13 tests + RAL)
make ral SIM=vcs           # RAL only on a commercial venue
# or directly:
cd lane2/lane2b_native/suite && ./run_commercial.sh vcs all 1
```

VCS name-resolution rule (important): commercial runs use the *_commercial.f filelists, which do **not** compile the Accellera uvm_pkg.sv — the vendor built-in UVM is used instead (-ntb_opts uvm-1.2 / -uvm / -L uvm). Compiling the Accellera source alongside the vendor UVM double-defines uvm_pkg and breaks VCS name resolution through the import chain. Every leaf package is imported directly (no re-export chain). Only the Verilator 5.050 path (ecpri.f, build_*_uvm.sh) compiles uvm_pkg.sv, because Verilator has no built-in UVM. See docs/VCS_PORTABILITY_NOTES.md for the scheduling/sampling-race analysis and harness status (Verilator-verified; VCS run pending).
All lanes together (incl. RAL)

```
make regress RAND=100      # lane1 + lane2a + lane2b + ral
-> combined_regression.json (1523/1523)
```

3. WAVEFORM DUMP (per simulator)

```
make waves SIM=<sim> COMPONENT=<slug> TEST=<t> SEED=<n>
```

Simulator	How	Format	Viewer
Verilator	--trace-fst (WAVES=1 / make waves SIM=verilator5050 ...)	FST	gtkwave
VCS	vcs - debug_access+all -kdb ... then UCLI dump -add /; run	VPD / FSDB	dve / verdi
Questa	vsim - voptargs+=+acc -do 'add wave -r /*; run -all'; wlf2vcd vsim.wlf -o waves.vcd	WLF / VCD	vsim / gtkwave
Xcelium	xrun -access +rwc -input @'database -open waves -shm; probe -create -	SHM / VCD	simvision

```
all -depth all;  
run'
```

Policy: waveforms are dumped on FAIL and preserved (never deleted).

4. COVERAGE REPORT (CODE + FUNCTIONAL, per simulator)

Verilator (code stmt/branch/toggle + functional + HTML dashboard)

```
make coverage  
# -> regression/code_cov.json          (line/branch/toggle;  
TB-defensive waivers)  
# -> regression/suite_regression_full.json (functional per  
component)  
# -> docs/COVERAGE_DASHBOARD.html      (open in a browser)  
# -> docs/FULL_COVERAGE_REPORT.md / .pdf
```

Mechanism: custom `sim_main.cpp` flushes `coverage.dat`; `code_cov.py` merges positive + negative + corrupt-golden runs; `report.py` merges functional COV-### bins and builds the HTML dashboard.

Commercial simulators (merge + HTML report)

```
make cov-report SIM=vcs          # urg -dir simv.vdb -report urgReport  
(HTML)  
make cov-report SIM=questa      # vcover merge merged.ucdb *.ucdb ;  
vcover report -html -htmldir covhtml merged.ucdb  
make cov-report SIM=xcelium     # imc: merge cov_work/scope/* -out  
merged ; load ; report -html -out covhtml
```

Compile with coverage first: VCS -cm line+cond+fsm+branch+tgl+assert (+ -cm at run); Questa +cover -coveropt 3; Xcelium -coverage all. Native coverage (Lane-2B) with cross + illegal_bins is sampled in-simulator on all UVM-capable simulators.

5. Documents (docs/ and docs/pdf/)

Verification Reference (VRD), Functional Design (FD), Integration Guide, Cross-Simulator & UVM README, Suite Guide, Architecture, Full Regression Report, Full Coverage Report, Signoff — Markdown + A4 PDF. HTML coverage dashboard: `docs/COVERAGE_DASHBOARD.html`.

6. Reproduce end-to-end

```
make verilator050 && make uvm-setup && make regress RAND=100 && make  
coverage
```

7. Open items (non-blocking)

GATE-0 requirements DERIVED (pin O-RAN.WG4.CUS R004-v16 -> authoritative). Commercial-sim Lane-2B/RAL portability optional (same UVM source). Combined executed: **1523/1523 = 100%**.